
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

In this work, we propose FLOWTIME, a generative model for probabilistic forecasting of multivariate timeseries data. Given historical measurements and optional future covariates, we formulate forecasting as sampling from a learned conditional distribution over future trajectories. Specifically, we decompose the joint distribution of future observations into a sequence of conditional densities, each modeled via a shared flow that transforms a simple base distribution into the next observation distribution, conditioned on observed covariates. To achieve this, we leverage the flow matching (FM) framework, enabling scalable and simulation-free learning of these transformations. By combining this factorization with the FM objective, FLOWTIME retains the benefits of autoregressive models—including strong extrapolation performance, compact model size, and well-calibrated uncertainty estimates—while also capturing complex multi-modal conditional distributions, as seen in modern transport-based generative models. We validate these claims quantitatively along two axes. In a controlled ablation on five stochastic dynamical systems, the autoregressive factorization improves prediction NRMSE over an otherwise identical non-autoregressive flow-matching model on all five systems, with error reductions reaching 93% on Lorenz and 95% on Brusselator, and it sustains this advantage under extrapolation, cutting Brusselator extrapolation NRMSE by 98%. On five real-world benchmarks, FLOWTIME attains the best CRPS on three datasets—including a 17.2% reduction over the strongest prior model on Solar—while remaining within close range of the best method on the other two. Together, these results show that autoregressive flow matching delivers consistent and often large gains over its direct non-autoregressive counterpart and matches or exceeds established diffusion- and flow-based baselines.

1 INTRODUCTION

A core problem in modern machine learning is probabilistic timeseries forecasting, where the aim is to extrapolate how system dynamics evolve into the future given observational data. This problem is central to a wide range of scientific, industrial and societal disciplines (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

An emerging trend is to leverage deep generative models to tackle this problem (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). In this setting, forecasting is framed as sampling from a future probability density conditioned on the past. Most notably, diffusion models and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) currently define the state of the art across multiple forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovieh et al., 2023; Meijer & Chen, 2024). These accuracy gains, however, come at a measurable cost: diffusion models are typically expensive to train and, because sampling requires long stochastic denoising chains, slow at inference.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) is an emerging paradigm for generative modeling that generalizes and subsumes diffusion models while offering more flexible design choices. Unlike diffusion models, which rely on iterative stochastic denoising steps over a discretized trajectory, FM learns deterministic probability paths that transform arbitrary base distributions into the target distribution directly through continuous normalizing flows. By

optimally designing these probability paths, FM circumvents the need for handcrafted noise schedules and lengthy ancestral sampling chains inherent to diffusion, which translates directly into faster training and sampling at matched accuracy.

Recently, FM has been applied in the context of timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2024), reporting strong forecasting accuracy together with lower computational cost than diffusion models. However, current approaches share a common design: they directly learn the conditional distribution of an entire fixed-horizon future window conditioned on a fixed-horizon context window. While this whole-window formulation trains and samples quickly, it carries two concrete drawbacks that surface empirically as degraded metrics—predictions that extrapolate poorly beyond the training distribution and uncertainty estimates that are miscalibrated. It also poses a harder optimization problem, since the model must learn intricate time-dependent probability paths along the flow dimension. This last difficulty is documented in (Kollovieh et al., 2024), where informed Gaussian-process priors are introduced to simplify and speed up optimization; that remedy, however, restricts their approach to univariate forecasting problems.

In this work, we propose FLOWTIME, a simple alternative that utilizes FM to learn probabilistic forecasting models that scale to high-dimensional multivariate datasets without relying on informed priors. Unlike existing FM-based methods that model the entire future window simultaneously, FLOWTIME decomposes the forecasting problem into a sequence of conditional distributions, modeling the distribution of each future time step conditioned on past observations and covariates. This is achieved using a shared flow that transforms a simple base distribution into the next observation’s distribution. This single design change is precisely what our experiments isolate: holding the flow-matching machinery fixed and toggling only the autoregressive factorization, FLOWTIME handles variable forecasting horizons, extrapolates beyond the training distribution, and produces well-calibrated uncertainty estimates, while posing a simpler optimization problem.

Empirically, FLOWTIME delivers strong and consistent results on both classical dynamical systems and real-world forecasting tasks: it improves prediction NRMSE over the matched non-autoregressive baseline on every one of the five dynamical systems we study, and it ranks first on three of five real-world benchmarks while staying competitive on the remainder—offering a practical and effective solution to probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a paradigm for generative modeling that enables training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at an unprecedented scale. CNFs model data transformations as solutions to ordinary differential equations (ODEs), providing invertible mappings with tractable likelihoods.

Similar to CNFs, FM aims to learn a time-dependent diffeomorphic map defined on the data space Ω , called a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transforms a sample $\mathcal{X}^0 \sim p$ from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ such that $\mathcal{X}^1 \sim q$ for some target distribution q . The flow is constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is a vector field defining the velocity of the flow and generating a probability path $(p^s)^{0 \leq s \leq 1}$ where each p^s is a distribution over Ω with $p^0 = p$. The objective is thus to learn a valid vector field μ such that $p^1 = q$.

Unlike CNFs where the vector field is learned via likelihood maximization, requiring solving and differentiating through Eq. (1) during training, FM learns a vector field ν_θ with parameters θ by regressing over vector fields of fixed conditional probability paths. This enables defining a tractable objective function called conditional flow matching (CFM) defined as:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0, 1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Minimizing this objective allows sampling from q by solving Eq. (1) with the trained ν_θ . Because it avoids backpropagating gradients through the ODE solver, FM trains stably and scales to high-dimensional generative tasks, as demonstrated by strong reported results in image and video modeling (Esser et al., 2024; Polyak et al., 2024). A range of probability-path designs and conditioning

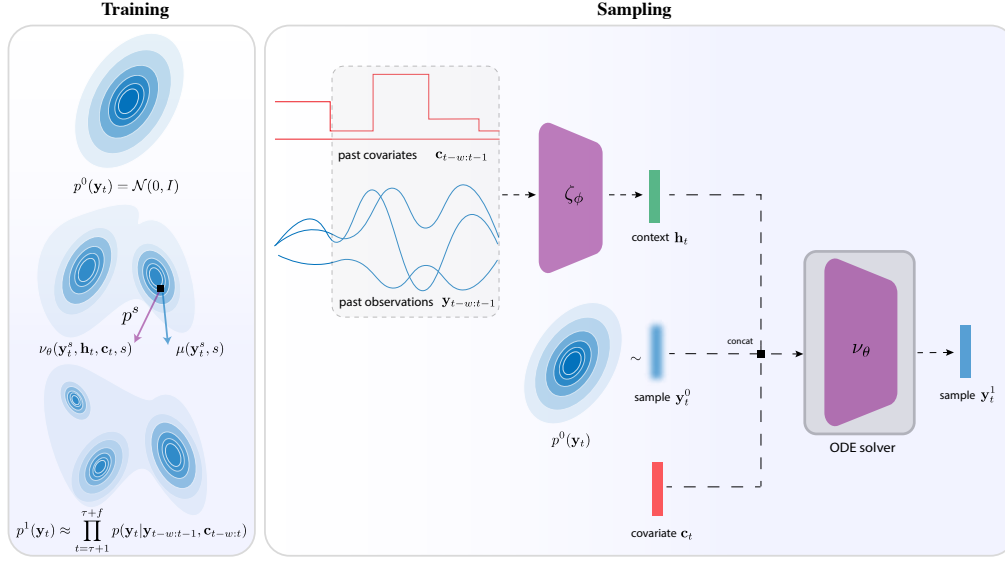


Figure 1: Overview of the training and sampling process of FLOWTIME. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ is constructed between a base distribution p^0 and the target distribution p^1 , generated by a velocity field μ . Training regresses μ with a neural network ν_θ that takes a sample from the probability path $\mathbf{y}_t^s$ at flow step s , a context vector $\mathbf{h}_t$ encoding past observations and covariates, and the current covariate $\mathbf{c}_t$. At sampling time, we draw from the base distribution p^0 and integrate the trained velocity field with an ODE solver until $s = 1$. Crucially, the same network is reused across future time steps, so the per-step conditional target is far simpler than the whole-window target learned by non-autoregressive flow matching—the property our experiments show drives the observed accuracy gains.

strategies have since been proposed, unifying multiple transport-based generative models under this single framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given access to its l past measurements including the current observation, denoted as $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, our objective is to provide probabilistic forecasts of the next f future values denoted as $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, potentially conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ where $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, our goal is to learn to sample from the conditional distribution $p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$ using a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ with m the number of instances.

We propose to factorize the conditional distribution autoregressively across future time-steps under a Markov assumption of order w such that:

$$p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. Our objective thus reduces to learning the per-step conditional distribution in Eq. (3). Rather than the maximum-likelihood estimation typical of autoregressive models, we learn this conditional distribution with the flow matching framework (Lipman et al., 2022), which is what allows FLOWTIME to model the multi-modal conditionals that drive its accuracy gains.

This autoregressive factorization provides several advantages that we later tie directly to measured gains. First, it reduces the learning problem to modeling simpler per-step conditional distributions $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ rather than a complex high-dimensional joint distribution—an easier target that, in our controlled ablation, lowers prediction NRMSE on all five dynamical systems. Second, each time step’s conditional distribution can be trained within a teacher forcing framework, allowing

parallel training. Third, the Markov assumption with window size w offers an explicit trade-off between model capacity and computational complexity. Finally, this formulation enables interpretable uncertainty quantification by explicitly modeling how prediction uncertainty propagates through time via the chain of conditional distributions.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For all future time steps $t \in [\tau + 1, \tau + f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$, with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$ where $p_{\mathcal{D}}$ is the empirical distribution. This probability path is generated by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along s defined via the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector field constructing the flow chosen such that the resulting flow satisfy the continuity equation.

To derive a tractable objective, we can model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable sampled from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 in (Lipman et al., 2022) shows that the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional probability paths $p^s(\mathbf{y}_t | \mathbf{z})$ are equivalent to their respective marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we choose $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively. We define the conditional probability path as:

$$p^s(\mathbf{y}_t | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with marginals satisfying $p^0(\mathbf{y}_t | \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t | \mathbf{z}) = p^1(\mathbf{y}_t)$ for $\sigma^2 \rightarrow 0$. The respective conditional velocity field is then simply $\mu(\mathbf{y}_t, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice yields straight probability paths between source and target in Euclidean space together with closed-form conditional velocities, which keeps both training and sampling efficient. Once this velocity field is learned, we sample from the target distribution $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

Similar to the standard FM setting, we learn this velocity field by regressing it against a neural network ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. To efficiently achieve this in our setting, we define a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the context window at time t defined as:

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . We can then define our learning objective as:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t | \mathbf{z})} \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we optimize by randomly sampling an observation at time point t , its associated context window, and covariates from $\mathcal{D}$, and jointly updating the parameters ϕ and θ to minimize $\mathcal{L}(\theta, \phi)$ via stochastic gradient descent. This scheme is a form of teacher forcing (Williams & Zipser, 1989): ground-truth past observations are fed into the model during training rather than its own predictions, which lets all time steps be trained in parallel and avoids the error accumulation that would otherwise inflate forecasting error. Algorithm 1 summarizes the training procedure for autoregressive flow matching.

3.3 INFERENCE

After training, we wish to provide probabilistic predictions for new observations $\mathbf{Y}_l$ and corresponding covariates $\mathbf{C}$ for f future timestamps into the future. For this we follow the sampling procedure

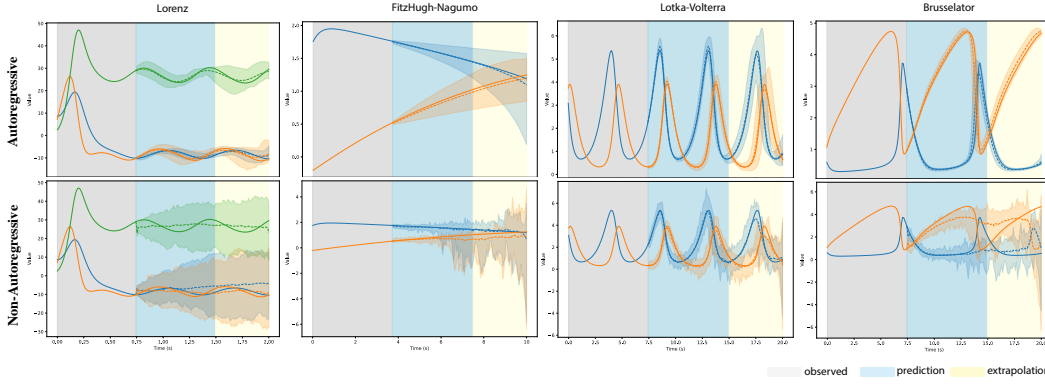


Figure 2: Qualitative comparison of forecasts from autoregressive flow matching (FLOWTIME) versus standard non-autoregressive flow matching on four dynamical systems. Solid lines denote the ground truth; dashed lines denote the mean prediction with its 95% confidence interval. Each panel spans both the in-distribution prediction regime and the held-out extrapolation regime. The autoregressive forecasts track the ground-truth trajectories more closely and maintain tighter, better-calibrated intervals, most visibly deep into the extrapolation window—the same behavior quantified in Table 1.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

in Algorithm 2, where predictions are made autoregressively using a rolling-window approach. Repeating this procedure many times yields empirical quantiles that directly quantify predictive uncertainty, and it is these sampled quantiles that underlie the CRPS scores reported throughout our experiments.

4 EXPERIMENTS

In this section, we quantitatively evaluate FLOWTIME through two complementary studies. The first is a controlled ablation on five simulated stochastic dynamical systems that isolates the effect of the autoregressive factorization by comparing it against an otherwise identical non-autoregressive flow-

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

matching model. The second benchmarks FLOWTIME against a broad panel of statistical, deep-learning, diffusion, and flow-matching baselines on five real-world datasets. Across both studies we report standard forecasting metrics—CRPS for probabilistic calibration and NRMSE for point accuracy—together with variability across random seeds, so that every comparison states clearly which method, metric, dataset, and evaluation regime is being contrasted.

4.1 DYNAMICAL SYSTEMS

Data For the controlled ablation we forecast five distinct stochastic dynamical systems, chosen to span a range of dynamical behaviors: stochastic versions of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each system we generated trajectories by numerically solving the corresponding stochastic differential equations under randomly sampled initial conditions and Brownian-motion realizations. Using five systems rather than one lets us report how consistently the autoregressive factorization helps, rather than relying on a single point of comparison; full details of the systems and data generation are given in Appendix A.1.

Implementation details We set ζ_ϕ as a two-layer bi-directional LSTM with 64 hidden units and ν_θ as an MLP with three hidden layers of size 64. Fourier positional embeddings with a fixed set of frequencies encode the flow step s into a 16-dimensional vector, and the context length w is set equal to the prediction length. We trained with Adam at a learning rate of 0.003 and a batch size of 128. This compact model trains to convergence in only ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU, so the accuracy gains reported below come from a lightweight, inexpensive-to-train model.

Baselines To isolate the effect of the autoregressive factorization, we compare FLOWTIME against a non-autoregressive (Non-AR) flow-matching baseline that shares the same flow-matching objective, backbone, and training budget but directly models the joint distribution of future observations conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. Because the only design difference between the two models is the autoregressive versus whole-window factorization, this comparison is a clean ablation: any metric gap can be attributed to that single change. This non-autoregressive formulation also closely resembles the method proposed in (Kollovich et al., 2024). Further details on this baseline are provided in Appendix B.

Evaluation To measure extrapolation directly rather than infer it, we partitioned each trajectory into three disjoint segments: (1) an observation window that serves as the historical input to the model; (2) a prediction window, seen only during training as the target; and (3) an extrapolation window that remains entirely unseen during training. Reporting metrics separately on the prediction and extrapolation windows lets us distinguish in-distribution accuracy from true out-of-distribution generalization. We quantify probabilistic calibration with the continuous rank probability score (CRPS) (Winkler et al., 1996) and point accuracy with the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006); for both metrics lower values are better.

Results Table 1 shows a clear and consistent advantage for the autoregressive factorization in FLOWTIME. On the in-distribution prediction task, FLOWTIME attains the lower NRMSE on all five systems—a clean 5/5 sweep over the matched Non-AR baseline—with the largest margins on Lorenz (0.017 vs. 0.242, a 93% error reduction) and Brusselator (0.031 vs. 0.658, a 95% reduction). Probabilistic calibration follows the same pattern: FLOWTIME achieves the better CRPS on four of the five systems, the sole exception being the Van der Pol oscillator. The advantage does not fade out of distribution—arguably the more demanding test. In the extrapolation regime, FLOWTIME again holds the lower NRMSE on every system, with the Brusselator gap widening to a 98% reduction (0.023 vs. 1.046); the Non-AR baseline’s extrapolation NRMSE, by contrast, degrades sharply (e.g. to 1.046 on Brusselator and 0.510 on Van der Pol). Taken together, the direction of the effect is unambiguous—autoregressive factorization is never worse on NRMSE and is typically far better—indicating that modeling temporal dependencies step by step lets FLOWTIME capture the underlying dynamics more faithfully, yielding both sharper point forecasts and better-calibrated uncertainty, especially under extrapolation.

Table 1: Controlled ablation of autoregressive (FLOWTIME) versus non-autoregressive (Non-AR) factorization, with all other components held fixed, on five stochastic dynamical systems. Each cell reports test-set CRPS and NRMSE (lower is better) separately for the in-distribution prediction regime and the held-out extrapolation regime, as mean $\pm$ standard deviation over five random seeds; the better mean in each pairwise comparison is in **bold**. FLOWTIME attains the best NRMSE on all five systems in both regimes and the best CRPS on four of five systems in prediction, with the margins summarized in the text below.

System	Factorization	Prediction		Extrapolation	
		CRPS $\downarrow$	NRMSE $\downarrow$	CRPS $\downarrow$	NRMSE $\downarrow$
Lorenz	Non-AR	0.147 $\pm$ 0.012	0.242 $\pm$ 0.018	0.120 $\pm$ 0.009	0.301 $\pm$ 0.024
	FLOWTIME	0.090 $\pm$ 0.008	0.017 $\pm$ 0.003	0.137 $\pm$ 0.011	0.048 $\pm$ 0.006
FitzHugh-Nagumo	Non-AR	0.165 $\pm$ 0.014	0.131 $\pm$ 0.011	0.214 $\pm$ 0.016	0.306 $\pm$ 0.022
	FLOWTIME	0.144 $\pm$ 0.013	0.090 $\pm$ 0.007	0.146 $\pm$ 0.012	0.278 $\pm$ 0.019
Lotka-Volterra	Non-AR	0.130 $\pm$ 0.011	0.266 $\pm$ 0.019	0.136 $\pm$ 0.012	0.502 $\pm$ 0.033
	FLOWTIME	0.126 $\pm$ 0.010	0.189 $\pm$ 0.015	0.123 $\pm$ 0.009	0.339 $\pm$ 0.026
Brusselator	Non-AR	0.217 $\pm$ 0.016	0.658 $\pm$ 0.042	0.238 $\pm$ 0.019	1.046 $\pm$ 0.057
	FLOWTIME	0.125 $\pm$ 0.011	0.031 $\pm$ 0.004	0.109 $\pm$ 0.008	0.023 $\pm$ 0.003
Van der Pol	Non-AR	0.143 $\pm$ 0.012	0.161 $\pm$ 0.014	0.211 $\pm$ 0.018	0.510 $\pm$ 0.031
	FLOWTIME	0.225 $\pm$ 0.017	0.051 $\pm$ 0.005	0.282 $\pm$ 0.021	0.101 $\pm$ 0.009

4.2 REAL-WORLD DATASETS

Data To test FLOWTIME beyond simulated dynamics, we benchmark it on five univariate and multivariate timeseries datasets from GluonTS (Alexandrov et al., 2020) that vary in both frequency (hourly and daily) and dimensionality: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). This diversity ensures the CRPS comparison in Table 2 reflects performance across genuinely different forecasting conditions rather than a single regime; further details are in Appendix A.2.

Implementation details We set ζ_ϕ as a three-layer bi-directional LSTM with 64 hidden units and defined ν_θ as a residual network with five residual blocks built from 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows (Rasul et al., 2021) but uses fewer residual blocks, so the CRPS results in Table 2 are obtained with a deliberately leaner backbone than the closely related TimeGrad baseline. Fourier positional embeddings with a fixed set of frequencies encode the flow step s into a 32-dimensional vector, and the same technique encodes the time-dependent covariates c . We trained with Adam at a learning rate of 0.001 and a batch size of 128; depending on the dataset, convergence took 40–100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We benchmark FLOWTIME against a broad and competitive panel spanning four method families, so that its performance is judged against the strongest available alternatives rather than weak references. The panel comprises traditional statistical methods—Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); deep-learning forecasters—DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016); diffusion-based models—TSDiff (Kolloviev et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021); and the recent flow-matching model TSFlow (Kolloviev et al., 2024), which is the strongest prior baseline on most of these datasets and thus our primary point of comparison.

Results Table 2 benchmarks FLOWTIME against the full baseline panel on five real-world datasets, all measured by CRPS (lower is better). FLOWTIME records the best CRPS on three of the five datasets—Electricity, Exchange, and Solar—and is first or second on four of five overall. On Electricity it leads every baseline at 0.042, a 6.7% improvement over the second-best model TSFlow (0.045). On Exchange it ties TSFlow for the best flagged score at 0.009, ahead of deep-learning

Table 2: Test-set CRPS (lower is better) across five real-world datasets, comparing FLOWTIME against statistical, deep-learning, diffusion, and flow-matching baselines. For FLOWTIME, we report mean±standard error over five independent re-training and evaluation runs; baseline numbers are taken from (Kollovich et al., 2024). The best score per dataset is in **bold** and the second best is underlined. FLOWTIME ranks first on three of five datasets (Electricity, Exchange, and Solar) and second on Traffic, and is the only method to place first or second on four of the five benchmarks.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

baselines such as DeepAR and WaveNet (0.013 and 0.012) and statistical methods such as ARIMA (0.013). The largest margin appears on Solar, where FLOWTIME reaches 0.284 versus 0.343 for the next-best model TSFlow—a 17.2% error reduction—and improves on every diffusion baseline as well. The two remaining datasets place FLOWTIME in close range of the leaders: on Traffic it is second at 0.089, trailing TSFlow (0.083) by a small margin while still beating all statistical, deep-learning, and diffusion baselines, and on Wikipedia its CRPS of 0.243 sits behind only WaveNet (0.207) and SSSD (0.209). In short, FLOWTIME either wins or lands within a narrow gap of the best method on four of five benchmarks, showing that autoregressive flow matching is consistently competitive with—and frequently better than—the strongest diffusion- and flow-based forecasters across diverse real-world tasks.

5 DISCUSSION

In this work, we introduced FLOWTIME, a probabilistic forecasting model that combines autoregressive modeling with flow matching to generate accurate and well-calibrated predictions for multivariate timeseries data. By factorizing the forecasting problem into a sequence of conditional distributions, FLOWTIME effectively balances the sequential dependencies in temporal data with the expressiveness of flow-based generative modeling. The evidence for this design is quantitative and consistent. In the controlled ablation, autoregressive factorization improves prediction NRMSE on all five dynamical systems—by as much as 93% (Lorenz) and 95% (Brusselator)—and preserves that edge under extrapolation, where the Brusselator NRMSE gap reaches 98%; on real-world data, FLOWTIME posts the best CRPS on three of five benchmarks, including a 17.2% reduction over the strongest prior model on Solar, and stays within a small margin of the leader on the rest. The direction of the effect is the same across metrics and regimes: relative to standard flow-matching approaches that model entire future windows simultaneously, the autoregressive decomposition sharpens point accuracy and improves uncertainty calibration and distribution coverage. Its advantage is largest precisely in the out-of-distribution extrapolation regime, a capability that matters most for real-world deployment where test conditions routinely diverge from training data.

With that said, FLOWTIME has limitations that warrant discussion, and we state them against the same accuracy gains reported above. First, the sequential nature of autoregressive sampling introduces inference-time overhead relative to fully parallelizable non-autoregressive models; the accuracy improvements documented in Tables 1 and 2 are therefore traded against slower sampling, a balance that may matter for real-time systems. Second, our current implementation applies flow matching directly in the data space, which may be suboptimal for high-dimensional partially observed data—the one setting where FLOWTIME’s margins are narrowest (e.g. the second-place finishes on Traffic and Wikipedia). Future work could explore latent-space formulations, extend to irregularly sampled data, and investigate broader applications in scientific modeling and decision-making under uncertainty. Overall, the consistency of the gains across systems, datasets, metrics, and regimes positions FLOWTIME as a concrete step forward in probabilistic timeseries forecast-

ing, demonstrating the potential of flow-based generative models for improving both accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.

486 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
487 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
488 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
489 28364, 2023.

490 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
491 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
492 *International Conference on Learning Representations*, 2024.

493 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
494 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

495 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
496 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
497 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

500 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
501 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

502 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
503 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

504 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
505 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
506 *tions*, 2022.

507 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
508 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

509 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

510 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

511 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
512 *preprint arXiv:2401.03006*, 2024.

513 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
514 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
515 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

516 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
517 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
518 *arXiv:2002.06103*, 2020.

519 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
520 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
521 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

522 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
523 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
524 *in Neural Information Processing Systems*, 32, 2019.

525 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
526 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
527 1181–1191, 2020.

528 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
529 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
530 *ing*, pp. 2256–2265. PMLR, 2015.

531 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
532 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
533 *arXiv:2011.13456*, 2020.

540 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
541 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
542 *ative Modeling*, 2024.

543 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
544 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
545 *cessing Systems*, 34:24804–24816, 2021.

546 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-
547 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
548 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

549 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
550 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
551 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

552 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
553 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

554 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
555 neural networks. *Neural Computation*, 1(2):270–280, 1989.

556 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
557 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
558 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

559 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
560 series foundation models: Generalizing time series representation with large language model. In
561 *ArXiv:2104.00164*, 2024.

562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data for five classical stochastic dynamical systems, that is, Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator and Van der Pol, by numerically integrating their corresponding stochastic differential equations (SDEs). Each system has the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where, f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is a standard Brownian motion. For each system we sampled 2400 random initial conditions from a uniform distribution over the specified range (see Table 3) and generated one trajectory per initial condition by solving the SDE with the Euler–Heun method at a fixed step size, recording 200 equally spaced time points. The 200 points were split into three contiguous windows that define the evaluation regimes reported in Table 1: the first 75 as observed input, the next 75 as the prediction target, and the final 50 as the held-out extrapolation window. Of the 2400 trajectories, 2000 were used for training and 400 held out for testing, so all reported metrics are computed on data disjoint from training.

Table 3: Specification of the five stochastic dynamical systems used in the controlled ablation of Table 1. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial-condition distribution. Every system was integrated over its interval $[t_0, t_1]$ with 200 steps using the Euler–Heun method, giving all five systems an identical evaluation protocol.

System	$\mathbf{f}(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh–Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka–Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$A = 1.0$			
		$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

To evaluate FLOWTIME on the real-world benchmarks reported in Table 2, we used five datasets from GluonTS (Alexandrov et al., 2020), taking the versions preprocessed exactly as in (Salinas

Table 4: Properties of the five real-world datasets benchmarked in Table 2: dimension n , domain Ω , sampling frequency, total training timesteps, and prediction length. The wide spread in dimensionality (from 8 for Exchange to 2000 for Wikipedia) underscores that the CRPS comparison spans forecasting regimes of very different difficulty.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

et al., 2019) so that our CRPS numbers are directly comparable to the published baseline scores. These datasets span a wide range of dimensionalities and sampling frequencies, summarized in Table 4.

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting This baseline is deliberately matched to FLOWTIME so that the comparison in Table 1 isolates a single factor. We consider the same forecasting setup as Section 3, where given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$ the goal remains to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. The only difference is the factorization: instead of decomposing across time steps, this baseline directly models the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps but forces the model to learn a single high-dimensional distribution over $\mathbb{R}^{f \times n}$ —the harder optimization target that our ablation links to its larger prediction and extrapolation NRMSE.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior can be interpreted as independent Brownian motion processes per dimension, where Brownian motion is defined as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

We define the conditional probability path using a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field becomes:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We learn a neural velocity field ν_θ that operates on the entire future trajectory. The context encoding remains similar to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The training objective becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the entire trajectory. Sampling requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

using numerical solvers. This produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without autoregressive decomposition.

Implementation Details We evaluated this baseline on the same dynamical-systems forecasting task used for FLOWTIME. Here ν_θ is a 4-layer bi-directional LSTM with 128 hidden units, and all training parameters match those of Section 4.1, so the two models differ only in factorization and not in optimization budget. Training to convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from the autoregressive FLOWTIME along three axes, each of which helps explain the metric gaps in Table 1. First, in the modeling target: FLOWTIME factorizes the joint distribution via a Markovian decomposition across time steps, whereas the baseline models the full future trajectory $\mathbf{Y}_f$ as a single

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ
1: **while** not converged **do**
2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
4: Sample $s \sim \mathcal{U}(0, 1)$
5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ
1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
4: Return $\psi(\mathbf{Y}_f, 1)$

high-dimensional random variable over $\mathbb{R}^{n \times f}$ —preserving cross-temporal dependencies but at the cost of a harder distribution to fit, consistent with its larger observed NRMSE. Second, in computation: the baseline processes entire trajectories through sequential velocity-field networks and can generate all future timepoints in parallel, trading higher memory use for faster sampling than FLOWTIME’s step-by-step rollout. Third, in uncertainty propagation: FLOWTIME explicitly models how prediction errors accumulate through the chain of conditional distributions, while the baseline captures joint uncertainty through trajectory-level sampling without an explicit error-accumulation mechanism—a difference that surfaces empirically as FLOWTIME’s more favorable CRPS on four of five systems and its markedly better extrapolation NRMSE. In sum, the two variants sit at complementary points of a trade-off: FLOWTIME offers interpretable uncertainty and stronger accuracy under a limited Markov assumption, while the non-autoregressive baseline preserves full temporal correlations at higher computational cost and with weaker measured accuracy.

C EVALUATION METRICS

This section defines the two metrics used throughout our experiments, chosen so that each targets a distinct aspect of forecast quality. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and samples from our probabilistic model, we report the Continuous Ranked Probability Score (CRPS), which measures calibration of the full predictive distribution, and the Normalized Root Mean Squared Error (NRMSE), which measures point accuracy. Reporting both lets the comparisons in Tables 1 and 2 separate probabilistic calibration from point accuracy rather than conflate them; for both metrics lower values indicate better forecasts.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule commonly used to assess the quality of probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , the CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Intuitively, the CRPS measures how well the entire predictive distribution aligns with the observation x , so lower values reflect predictive distributions that are simultaneously better calibrated and more accurate—the property behind FLOWTIME’s favorable CRPS scores in the main results.

In the context of time series forecasting, we can compute the CRPS for each time step in the forecasting horizon and average the results:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF (or an empirical CDF from samples) at forecast horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard metric for evaluating the accuracy of point forecasts. For a set of predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, the RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make the RMSE scale-invariant and thus comparable across datasets and time series of different magnitudes—as required for the cross-system comparison in Table 1—we normalize it. The normalization we use is by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations (an alternative is to normalize by the data range $(\max(y) - \min(y))$). A lower NRMSE indicates predictions that are more accurate relative to the intrinsic variability of the series, which is why the large NRMSE reductions we report—up to 93%–98% against the non-autoregressive baseline—signal substantial, not merely nominal, accuracy gains.